

Abstract classes in C++

BIE-PA2 – Programming and Algorithmic II

Ladislav Vagner

Department of Theoretical Computer Science
Faculty of Information Technology
Czech Technical University in Prague
`xvagner@fit.cvut.cz`

April 13, 2026

Overview

- Abstract classes.
- RTTI.
- Heterogeneous (polymorphic) data structures.

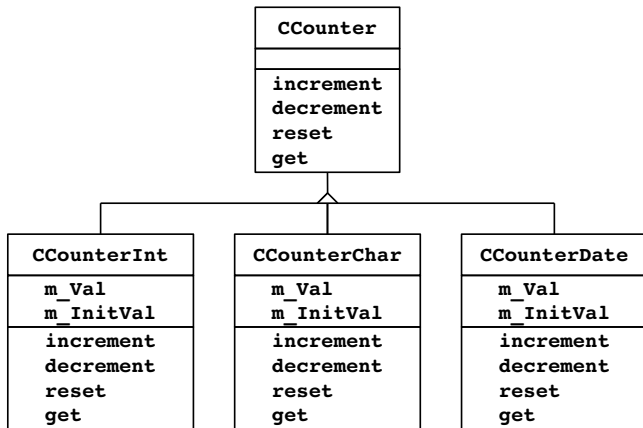
Abstract classes

- A counter may count values of different types:

	value	inc	dec
intCnt	10	11	9
charCnt	h	i	g
dateCnt	2011-04-11	2011-04-12	2011-04-10

- We will develop class CCounter implementing such a counter.
- The class will have no member variables, it only declares methods.

Abstract classes



Abstract classes

```
// counter.h
#ifndef PA2_COUNTER_H
#define PA2_COUNTER_H
#include <string>
struct CCounter
{
    CCounter() { }
    virtual ~CCounter() { }
    virtual void increment() = 0;
    virtual void decrement() = 0;
    virtual void reset() = 0;
    virtual std::string get() const = 0;
};
#endif
```

The syntax (**virtual** and **=0**) declares an abstract method. The method will be defined in a subclass.

Abstract classes

```
// counterint.h
#ifndef PA2_COUNTERINT_H
#define PA2_COUNTERINT_H
#include "counter.h"

class CCounterInt : public CCounter
{
    int m_Val, m_InitVal;
public:
    CCounterInt( int val) : m_InitVal(val) { reset (); }
    void increment() override { m_Val ++; }
    void decrement() override { m_Val --; }
    void reset()      override { m_Val = m_InitVal; }
    std::string get() const override {
        std::ostringstream os;
        os << m_Val; return os.str(); }
};
#endif
```

Abstract classes

```
// counterchar.h
#ifndef PA2_COUNTERCHAR_H
#define PA2_COUNTERCHAR_H
#include <cctype>
#include "counter.h"
class CCounterChar : public CCounter {
    char m_Val, m_InitVal;
public:
    CCounterChar( char val) : m_InitVal(val) { reset (); }
    void increment() override { m_Val ++; }
    void decrement() override { m_Val --; }
    void reset() override { m_Val = m_InitVal; }
    std::string get() const override {
        std::ostringstream os;
        if(std::isprint(m_Val)) os << m_Val; else
            os << "\\\" <<(int)m_Val; return os.str ();
    }
};
#endif
```

Abstract classes

```
// funwithcounter.cpp

static int menu() { ... }

void funWithCounter(CCounter * pc)
{
    while(true) {
        cout << "Value = " << pc->get() << endl;
        switch(menu()) {
            case 1: pc->increment(); break;
            case 2: pc->decrement(); break;
            case 3: pc->reset(); break;
            case 0: cout << "end" << endl;
                    return;
        }
    }
}
```


Abstract classes

```
// main.cpp

#include <iostream>
#include "counterint.h"
#include "counterchar.h"
#include "funwithcounter.h"
using namespace std;
int main() {
    CCounterInt    ic(0);
    CCounterChar  cc('A');

    cout << "Integer counting" << endl;
    funWithCounter(&ic);

    cout << "Character counting" << endl;
    funWithCounter(&cc);

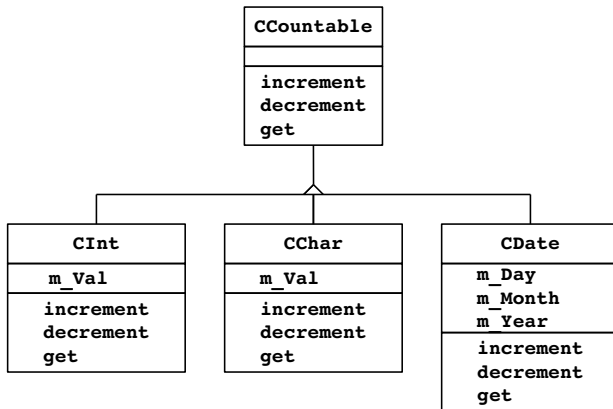
    return 0;
}
```

Abstract classes

- Abstract class is a class where method (methods) are declared, however the implementation is missing (set to 0).
- Abstract methods define the interface of a class. All subclasses will implement the interface.
- Abstract methods are useful if:
 - it is known that all subclasses will have the property (i.e. will be able to provide the implementation),
 - the implementation in a superclass does not make a sense (e.g. some important member variables are not present),
 - some unified view of the subclasses is required (e.g. a collection of instances in some container).
- Abstract class cannot have instances. Why?
- Pointers and references to abstract classes may be declared. Why?

Solution #2

We will introduce class CCountable – an abstract base class of countable values:



Solution #2

- The class CCounter will not be an abstract class.
- The class will be a value holder – it will reference CCountable subclasses.

CCounter
CCountable * m_Val CCountable * m_InitVal
increment decrement reset get

Why m_Val and m_InitVal is a pointer? Why not a value?

Solution #2

```
// counter.h
#ifndef PA2_COUNTER_H
#define PA2_COUNTER_H
#include <string>
#include "countable.h"

class CCounter {
    CCountable * m_Val;
    CCountable * m_InitVal;
public:
    CCounter(CCountable * val) : m_InitVal (val) { reset(); }
    void increment()      { m_Val->increment (); }
    void decrement()      { m_Val->decrement (); }
    void reset()          { m_Val = m_InitVal; }
    std::string get () const { return m_Val->get (); }
};
#endif
```

Solution #2

```
// countable.h
struct CCountable {
    virtual ~CCountable () { }
    virtual void increment() = 0;
    virtual void decrement() = 0;
    virtual std::string get() const = 0;
};

class CInt : public CCountable {
    int m_Val;
public:
    CInt(int val) : m_Val (val) { }
    void increment () override { m_Val++; }
    void decrement () override { m_Val--; }
    std::string get () const override {
        std::ostringstream os;
        os << m_Val; return os.str ();
    }
};
```

Solution #2

```
...  
int main()  
{  
    CInt      * iv = new CInt(0);  
    CChar     * cv = new CChar('A');  
    CCounter  intCnt( iv);  
    CCounter  charCnt( cv);  
  
    cout << "Counter int" << endl;  
    funWithCounter(&intCnt);  
    cout << "Counter char" << endl;  
    funWithCounter(&charCnt);  
    delete iv;  
    delete cv;  
    return 0;  
}
```

Solution #2

```
...  
int main()  
{  
    CInt      * iv = new CInt(0);  
    CChar     * cv = new CChar('A');  
    CCounter  intCnt( iv);  
    CCounter  charCnt( cv);  
  
    cout << "Counter int" << endl;  
    funWithCounter(&intCnt);  
    cout << "Counter char" << endl;  
    funWithCounter(&charCnt);  
    delete iv;  
    delete cv;  
    return 0;  
}
```

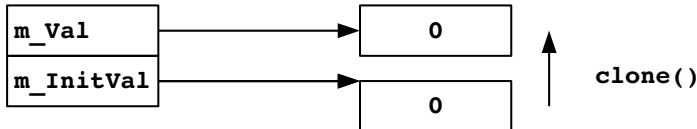
reset() does not work. Why?

Solution #2 fixup

- The problem is the `reset()` method.
 - the method copies the pointer from `m_InitVal` to `m_Val`,
 - thus both pointers reference the same object.

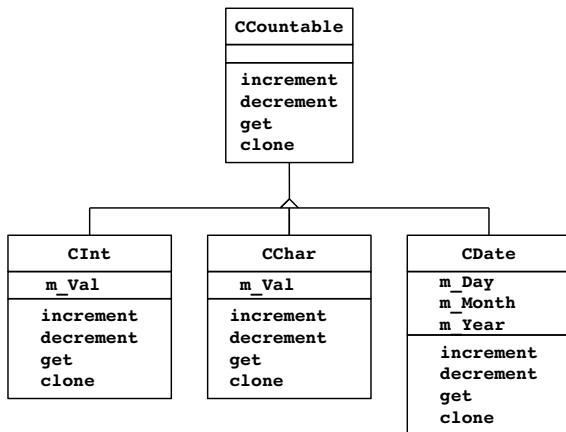


- Correct solution will prepare a new copy of the `m_InitVal` object and assign pointer to the copied object into `m_Val`:
 - we must copy the referenced object,
 - the referenced object may be any subclass of `CCountable`,
 - we introduce a new method `clone` in the `CCountable`.



Solution #2 fixup

Fixed class diagram (method clone):



Solution #2 fixup

```
// countable.h
struct CCountable {
    virtual ~CCountable () { }
    virtual void increment() = 0;
    virtual void decrement() = 0;
    virtual CCountable * clone () const = 0;
    virtual std::string get() const = 0;
};

class CInt : public CCountable {
    int m_Val;
public:
    CInt(int val) : m_Val (val) { }
    CInt * clone () const override { return new CInt(*this); }
    void increment () override { m_Val++; }
    void decrement () override { m_Val--; }
    std::string get () const override {
        std::ostringstream os;
        os << m_Val; return os.str (); }
};
```

Solution #2 fixup

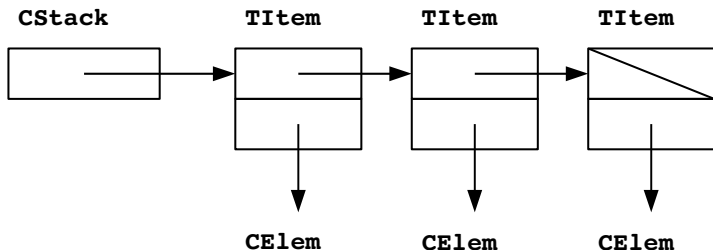
```
class CCounter {
    CCountable * m_Val;
    CCountable * m_InitVal;
public:
    CCounter(CCountable * val) : m_Val(nullptr),
                                m_InitVal(val)
    { reset (); }
    ~CCounter()                { delete m_Val; }
    void increment()           { m_Val->increment (); }
    void decrement()           { m_Val->decrement (); }
    void reset()               { delete m_Val;
                                m_Val = m_InitVal->clone ();
                                }
    std::string get () const { return m_Val->get (); }
};
```

Heterogeneous (polymorphic) data structures

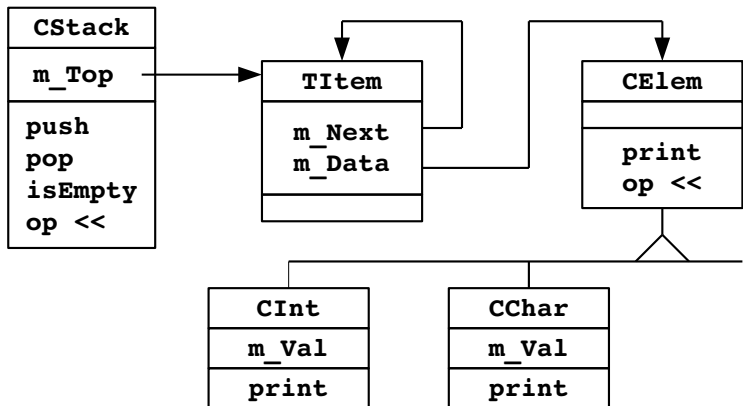
- Homogeneous data structures:
 - one data type stored in the data structure,
 - all elements in the data structure have the same interface,
 - implementation: generic class, generic parameter,
 - examples: stack of integers, stack of strings, ...
- Heterogeneous data structures:
 - elements of many different data types may be stored in the same data structure instance,
 - the elements usually share some common interface (common properties),
 - implementation: (abstract) base class of the elements, the base class gives the common interface.

Heterogeneous (polymorphic) stack

- The stack will be implemented in class `CStack` as a linked list.
- The items in the linked list will be structures `TItem`.
- Elements (values) stored in the stack will be subclasses of `CElem`.
- The class `CElem` will be an abstract base class.



Heterogeneous (polymorphic) stack



Heterogeneous (polymorphic) stack

```
// stack.h
#include <iostream>
class CElem {
public:
    virtual void print(std::ostream&) const = 0;
};

// no need to be a friend function
std::ostream& operator<<(std::ostream& s, const CElem& p)
{ p.print(s); return s; }

...
```


Heterogeneous (polymorphic) stack

```
...  
class CStack {  
    struct TItem {  
        CElem * m_Val;  
        TItem * m_Next;  
        TItem(CElem * v, TItem * n) : m_Val(v),  
                                     m_Next(n) { }  
    };  
    TItem * m_Top;  
public:  
    CStack() : m_Top(nullptr) { }  
    void push(CElem * v) {m_Top = new TItem(v, m_Top); }  
    CElem * pop();  
    int isEmpty() const { return m_Top == nullptr; }  
    friend std::ostream& operator<<( std::ostream&,  
                                     const CStack&  
                                     );  
};
```

Heterogeneous (polymorphic) stack

```
// stack.cpp
#include <iostream>
#include "stack.h"

CElem * CStack::pop() {
    if(! m_Top) return nullptr;
    TItem * tmp = m_Top;
    CElem * ret = m_Top->m_Val;
    m_Top = m_Top->m_Next;
    delete tmp;
    return ret;
}

std::ostream& operator<< (std::ostream& os, const CStack&s)
{ const CStack::TItem * tmp;
  for(tmp = s.m_Top; tmp; tmp = tmp->m_Next)
      os << *tmp->m_Val << std::endl;
  return os;
}
```

Heterogeneous (polymorphic) stack

```
#include "stack.h"

class CInt : public CElem {
    int m_Val;
public:
    CInt(int v) : m_Val(v) { }
    void print(ostream& os) const override { os << m_Val; }
};

class CChar : public CElem {
    char m_Val;
public:
    CChar(char v) : m_Val(v) { }
    void print(ostream& os) const override { os << m_Val; }
};
```

Heterogeneous (polymorphic) stack

```
void main()
{
    CStack s;
    CChar  cv( 'b');
    CInt    iv( 222), iv2(100);
    s.push(&iv);
    s.push(&cv);
    s.push(&iv2);
    cout << "dump:" << endl << s << endl;
    cout << "remove:" << endl;
    while (!s.isEmpty())
        cout << *s.pop() << endl;
    return 0;
}
```

Type casts

- The method `pop` returns a pointer to abstract class `CElem`.
- The value may be stored into `CElem * pointer`. An attempt to store the value into `CInt *` or `CChar *` causes a compile error.
- The error is reported even if the actual instance matches the expected return value type:

```
CStack s;  
  
s.Push(new CInt(10));  
s.Push(new CChar('A'));  
s.Push(new CChar('B'));  
  
CElem * e = s.pop (); // ok  
CChar * cp = s.pop (); // error  
CInt * ip = s.pop (); // error
```

Type casts

- The pointers may be casted using `(CChar*)/(CInt*)` type casts. This removes the compile error, however, the program may fail if the pointers (expected types) and the instances (actual types) do not match.
- Type checking may be enforced if `dynamic_cast` operator is used:
 - the operator checks the types in the run-time,
 - if the types match, the pointer is type casted,
 - if the types do not match, the operator returns `nullptr`.

Type casts

Example: use `dynamic_cast` to sum all integer values in the stack.

```
int main() {
    CStack s;
    CInt *pc1 = new CInt(10);    s.push(pc1);
    CInt *pc2 = new CInt(29);    s.push(pc2);
    CChar *pz = new CChar('a'); s.push(pz);
    cout << s << endl;
    CInt *pc;
    int sum = 0;
    while(! s.isEmpty()) {
        pc = dynamic_cast<CInt*>(s.pop());
        if(pc != nullptr) sum += pc->m_Val;
    }
    cout << "sum = " << sum << endl;
    delete pc1; delete pc2; delete pz;
    return 0;
}
```

RTTI

- RTTI (run time type info) is an additional information included in objects. Each object with RTTI knows its class in run-time.
- RTTI information is used by the `dynamic_cast` operator to decide whether the conversion is valid, or not.
- Object class may be examined in run-time by `typeid` operator.

RTTI

Example: use `typeid` to count `CInt`/`CChar` values.

```
#include <typeinfo> // important
int main() {
    CStack s;
    CElem *p;
    int ints = 0, chars = 0;
    // ... stack filled ...
    while (!s.isEmpty()) {
        p = s.pop();
        if (typeid(*p)==typeid(CInt))
            ints++;
        else if (typeid(*p)==typeid(CChar))
            chars++;
    }
    cout << "CInt: " << ints << endl;
    cout << "CChar: " << chars << endl;
    return 0;
}
```

RTTI

- RTTI may be useful, however, do not abuse it.
- Well designed classes seldom require `dynamic_cast` or `typeid`.
- Polymorphism attempts to simplify the high-level routine programming:
 - all objects understand a common interface,
 - each class implements the interface such that the semantic is preserved (e.g. each class knows `print`, the high-level routine simply calls `print` and the objects print themselves in their appropriate way),
 - complex `if/if else/switch` are avoided in the high-level routines.
- On the contrary, `dynamic_cast` and `typeid` introduce `if/if else/switch` in the high-level routines, effectively nullifying the benefits of polymorphism.
- `dynamic_cast` or `typeid` shall be considered the last resort before the entire program is redesigned and rewritten.

Questions ...